Integrantes	Matricula
Ivan Francisco Padron Macias	2150830
María Ximena Rodarte Cuevas	2154111
Ángel Haziel Bernal Garza	2156444

Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: Resumen
Fecha: 04/11/2026	
Bibliografía (Formato APA) Notas de clase URL del repositorio: https://github.com/xiro07/Unidad-5_CLN.git	

Reporte de evidencias – Unidad 5: Integración, Pruebas y Calidad

1. Objetivo y plan de pruebas

Este reporte demuestra, con pruebas automatizadas y evidencia reproducible, que las reglas de negocio del proyecto base de pedidos funcionan, que la persistencia y el cobro externo se comportan como se espera y que GitHub Actions detecta cambios defectuosos. Un programa que corre sin errores no prueba que sus reglas sean correctas: la evidencia sí.

Alcance. Proyecto Python `Proyecto_Python_Base_U5` (src/domain.py, application.py, repository.py, security.py), pytest, SQLite en memoria, `Mock` para la pasarela de pago, Ruff, pytest-cov y GitHub Actions. Los datos son ficticios.

Arquitectura. Usuario/API → Application Service (`OrderService`) → Entidad `Order` y reglas de dominio. Los puertos (repositorio y pasarela de pago) se

sustituyen en pruebas por SQLite en memoria y por un Mock. El dominio no importa SQLAlchemy ni bibliotecas de pagos.

Plan de pruebas: qué se prueba y qué se espera

Funcionalidad	Regla de negocio	Resultado esperado	Tipo de prueba
Confirmar pedido	Total mayor que cero	Order(0) y Order(-1) lanzan ValueError	Unitaria (error)
Confirmar pedido	Cliente con deuda no confirma	CustomerHasDebt	Unitaria (error)
Confirmar pedido	VIP recibe 15% de descuento	Order(200, vip=True) cobra 170; regular cobra 200	Unitaria (éxito)
Confirmar pedido	No confirmar dos veces	ValueError("already confirmed")	Unitaria (TDD)
Repositorio	Commit persiste; rollback descarta	Con commit el pedido existe; con rollback la consulta da None	Integración (SQLite)
Cobro	Pago aprobado o rechazado, sin cobros reales	Aprobado devuelve el importe; rechazado lanza RuntimeError	Integración (mock)
Cancelación	Enviado nunca; admin sí; dueño solo antes de 24 h	Matriz de autorización del Paso 6	Unitaria (seguridad)
Calidad y CI	Lint, pruebas y coberturas automáticas	Ruff sin errores, cobertura ≥80%, pipeline verde / rojo / verde	Cobertura y CI

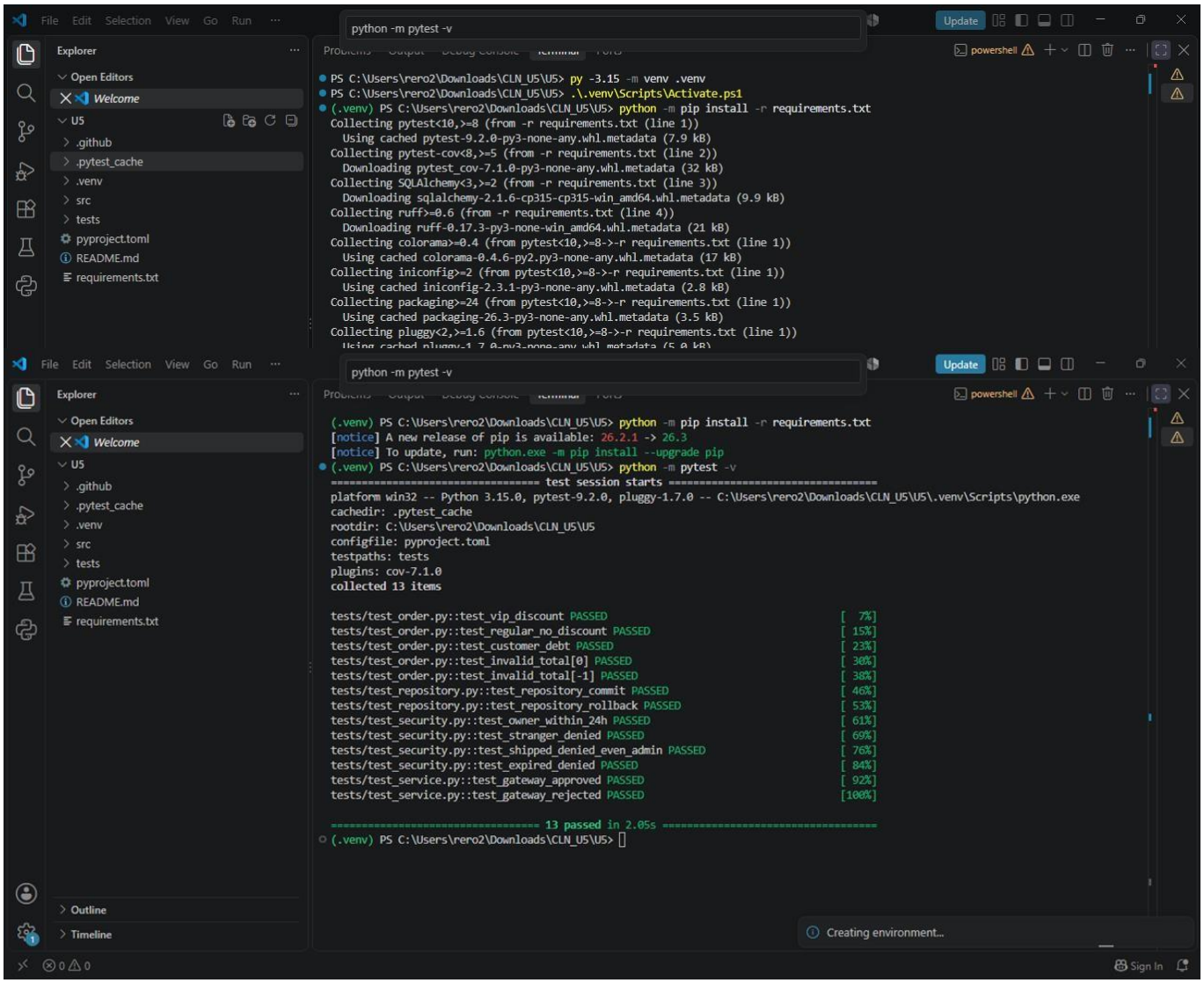
2. Paso 1 · Instalación y comprobación del proyecto (E1)

El proyecto base se instaló en un entorno virtual y sus pruebas se ejecutaron con pytest; todas terminaron en PASSED.

```
py -3.11 -m venv .venv                    # Windows (macOS/Linux: python3 -m venv
. venv)
.\.venv\Scripts\Activate.ps1              # macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -v
```

- `venv` crea un entorno aislado para las librerías del curso.
- `pip install -r requirements.txt` instala pytest, SQLAlchemy, Ruff y pytest-cov.
- `pytest -v` descubre las funciones `test_*`, las ejecuta y muestra el resultado de cada una.

Captura E1: terminal con el comando y el resumen de pruebas.



Explicación: El entorno virtual (venv) aísla las dependencias del proyecto para evitar conflictos con otras librerías del sistema y garantizar que el trabajo sea reproducible. Por su parte, pytest descubre y ejecuta automáticamente las funciones de prueba que comienzan con el prefijo test_. Esto permite verificar de forma automatizada y sin revisión manual que las reglas de negocio funcionan correctamente.

3. Paso 2 · Prueba unitaria y matriz de casos (E2)

Se añadieron dos pruebas unitarias a tests/test_order.py y se armó una matriz de 12 casos: 5 de éxito, 4 de error y 3 de límite. Cada prueba sigue el patrón Arrange-Act-Assert (preparar, ejecutar, verificar).

Dos nuevas pruebas unitarias agregadas al archivo dentro de test_order.py

```
# Paso 2: 2 pruebas extras

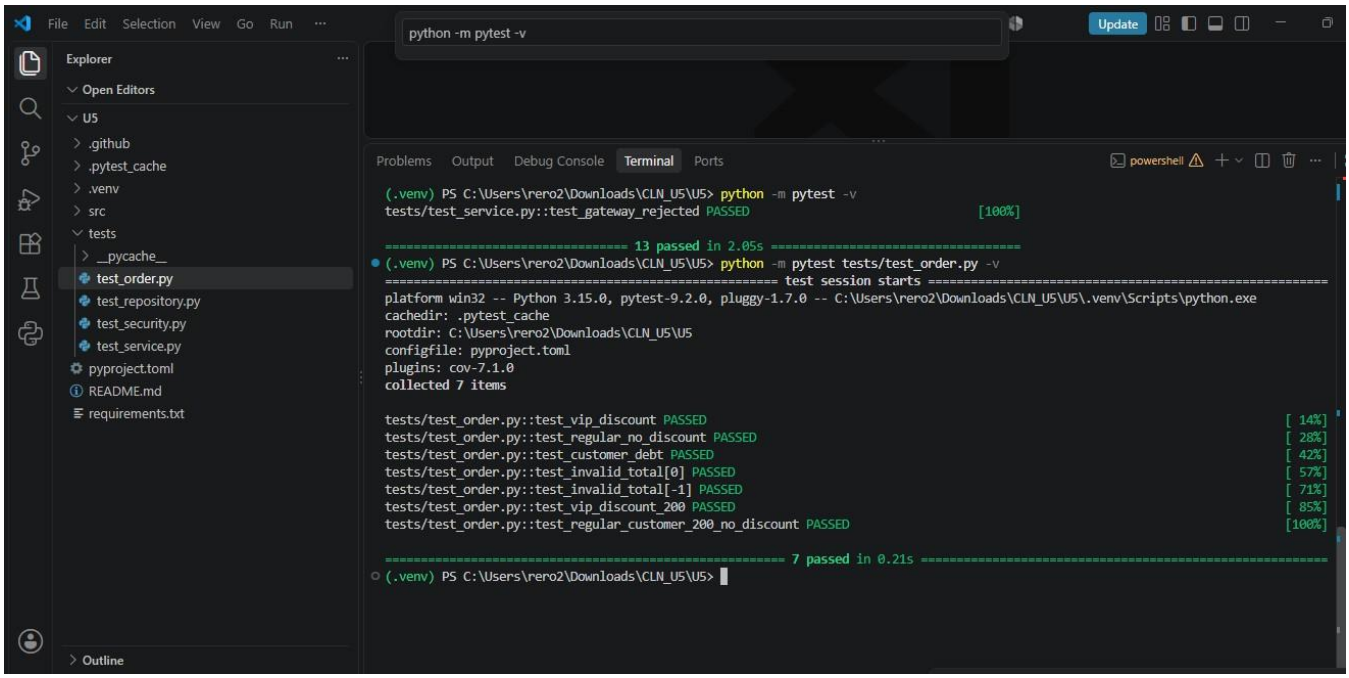
def test_vip_discount_200():
    o = Order(200, vip=True)
    assert o.confirm() == 170       # 200 - 15% (30) = 170

def test_regular_customer_200_no_discount():
    o = Order(200)
    assert o.confirm() == 200       # cliente regular: sin descuento
```

Se agregaron dos nuevas pruebas unitarias al archivo tests/test_order.py para validar diferentes escenarios de cobro. La primera prueba (test_vip_discount_200) verifica que a un pedido de $200 de un cliente VIP se le aplique correctamente el 15% de descuento, resultando en un cobro de $170. La segunda prueba (test_regular_customer_200_no_discount) comprueba que un cliente regular con un pedido de $200 pague el total exacto

sin ningún descuento. Ambas pruebas cumplen con el patrón Arrange-Act-Assert y finalizaron con éxito en el estado PASSED.

Captura E2: salida de las 7 pruebas de test_order.py en PASSED.



Matriz de casos de prueba

ID	Tipo	Regla	Entrada	Resultado Esperado
M01	Éxito	VIP 15%	Order(100, vip=True)	Cobro 85, estado CONFIRMED
M02	Éxito	Regular sin descuento	Order(100)	Cobro 100
M03	Éxito	VIP 15%	Order(200, vip=True)	Cobro 170
M04	Éxito	Regular sin descuento	Order(200)	Cobro 200
M05	Éxito	Pago aprobado	mock charge=True	Devuelve 100
M06	Error	Total positivo	Order(0)	ValueError
M07	Error	Total positivo	Order(-1)	ValueError
M08	Error	Deuda bloquea	confirm(customer_debt=50)	CustomerHasDebt
M09	Error	Pago rechazado	mock charge=False	RuntimeError
M10	Límite	Redondeo del descuento	Order(0.01, vip=True)	Cobro 0.01
M11	Límite	Deuda exactamente 0	confirm(customer_debt=0)	Se confirma (cobro 100)
M12	Límite	Cancelar a las 24 h exactas	dueño, antigüedad 24 h	Denegado

4. Paso 3 · TDD real: Red → Green → Refactor (E3)

La regla nueva “no confirmar un pedido ya confirmado” se implementó con TDD: la prueba se escribió y falló antes de tocar src/domain.py.

RED. Se agregó la prueba a tests/test_order.py y se ejecutó antes de modificar el código. Falló con DID NOT RAISE ValueError, porque la versión inicial permitía confirmar dos veces.

```
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

GREEN. Implementación mínima al inicio de confirm(); la prueba pasó y toda la suite siguió en verde:

```
if self.status == "CONFIRMED":
    raise ValueError("already confirmed")
```

REFACTOR. Las validaciones se agruparon en un método interno sin cambiar el comportamiento dentro de domain.py

```
@dataclass
class Order:
    total: float
    vip: bool = False
    status: str = "NEW"
    discount: float = 0.0

    def _ensure_can_confirm(self, customer_debt):
        if self.status == "CONFIRMED":
            raise ValueError("already confirmed")
```

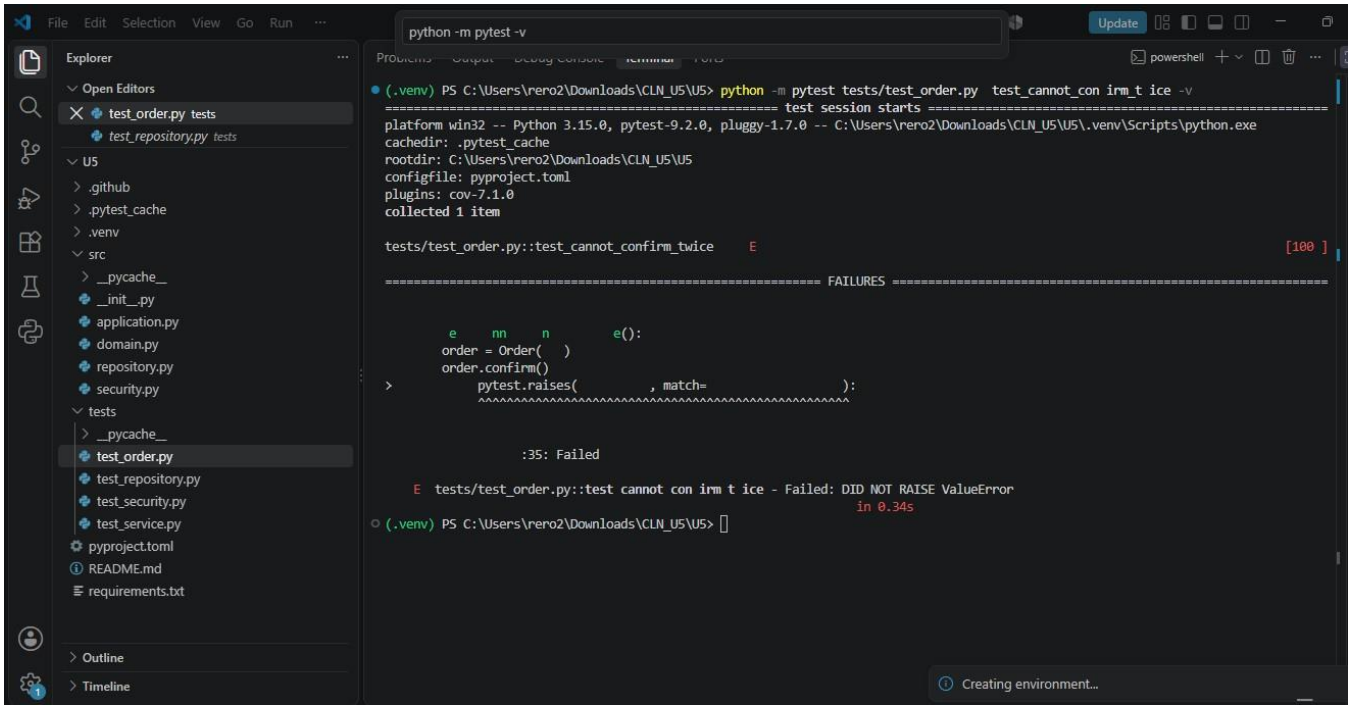
```
        if self.total <= 0:
            raise ValueError("total must be positive")
        if customer_debt > 0:
            raise CustomerHasDebt("outstanding debt")

    def confirm(self, customer_debt=0):
        self._ensure_can_confirm(customer_debt)
        self.discount = round(self.total * 0.15, 2) if self.vip else 0.0
        self.status = "CONFIRMED"
        return self.total - self.discount
```

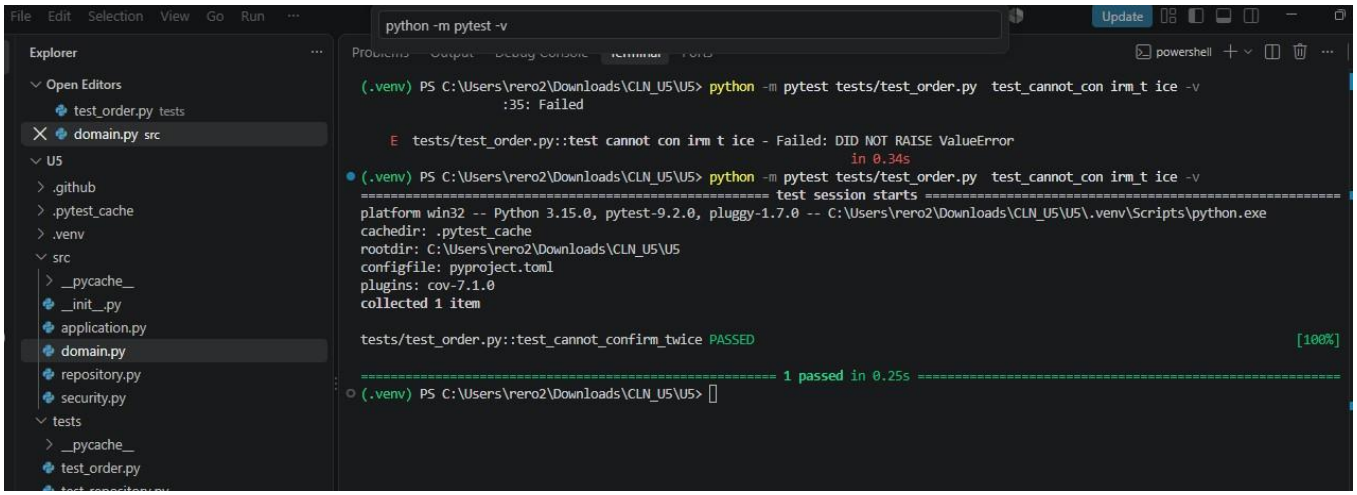
Cambio: El método _ensure_can_confirm agrupa las tres validaciones, y confirm() queda más corto y legible. El guion bajo indica que es un método interno.

Capturas E3:

Primera Prueba (RED)



Segunda prueba (GREEN)



¿Por qué TDD no es simplemente ejecutar pruebas después de programar?

En **TDD** la prueba actúa como un filtro inicial: se escribe antes de implementar la funcionalidad y se comprueba que falla, garantizando su efectividad. Escribir el código primero y las pruebas después impide verificar si el test realmente detecta fallos. Una vez comprobado el fallo, se desarrolla el código mínimo necesario para pasar a estado verde y se refactoriza bajo la red de seguridad de las pruebas.

5. Paso 4 · Integración con base de datos y transacciones (E4)

Se verificó que el repositorio guarda pedidos y respeta commit y rollback usando SQLite en memoria; cada prueba crea su propio motor, así que no se toca ninguna base real. `SqlOrderRepository.save()` agrega el pedido a la sesión y la sesión es el límite transaccional.

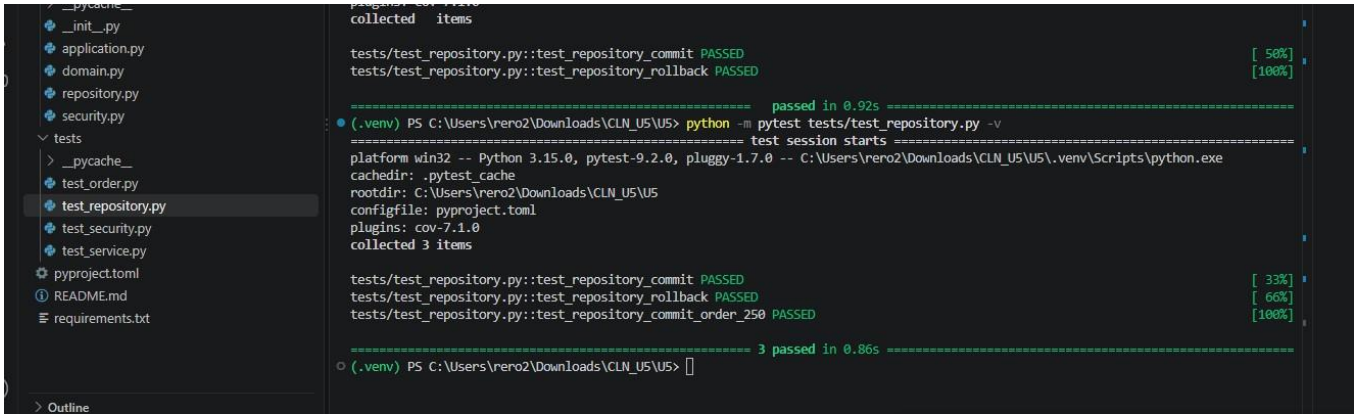
Código de la prueba dentro de test_repository.py

```
#Paso 4 del lab
def test_repository_commit_order_250():
    engine = make_engine()
    with Session(engine) as session:
```

```
    SqlOrderRepository(session).save(2, Order(250))
    session.commit()
    row = session.scalar(select(OrderRow).where(OrderRow.id == 2))
    assert row.id == 2
    assert row.total == 250
    assert row.status == "NEW"
```

La prueba guarda un pedido de $250 con id=2, hace commit y consulta la base: el registro existe con ese id, ese total y el estado inicial NEW. Se ejecutó con python -m pytest tests/test_repository.py -v.

Captura E4: las 3 pruebas del repositorio en PASSED.



¿Qué diferencia hay entre commit y rollback?

- commit: Confirma la transacción actual y hace permanentes los cambios en la base de datos.
- rollback: Revierte la transacción actual, deshaciendo los cambios y dejando la base de datos tal como estaba antes.

6. Paso 5 · Servicios externos sin cobros reales (E5)

El servicio de checkout se probó sin llamar a un proveedor de pagos: un Mock responde como si fuera la pasarela y no se transmite dinero. Con True, checkout devuelve el importe; con False, lanza RuntimeError("payment rejected").

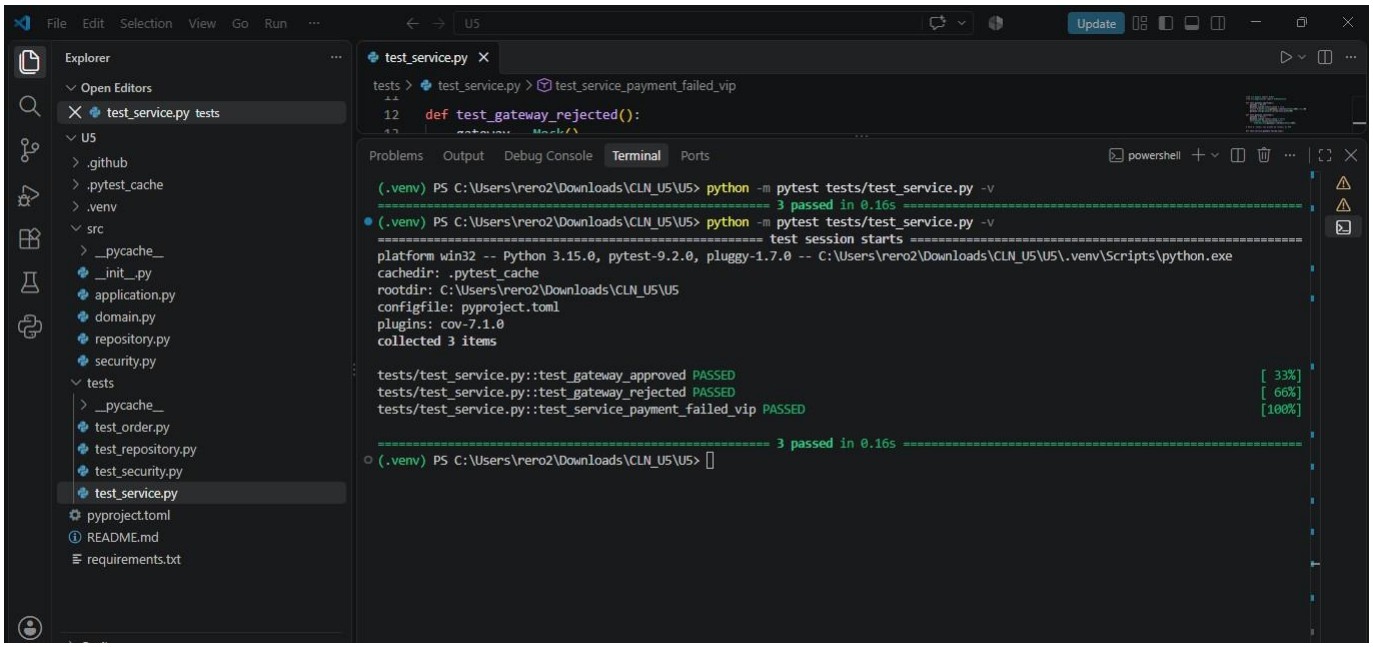
Código en test_service.py

```
# Paso 5: Añadir una prueba de rechazo en VIP

def test_service_payment_failed_vip():
    gateway = Mock()
    gateway.charge.return_value = False
    with pytest.raises(RuntimeError, match="payment rejected"):
        OrderService(gateway).checkout(Order(200, vip=True))
    gateway.charge.assert_called_once_with(170)
```

La prueba comprueba dos cosas: que el rechazo lanza RuntimeError y que el intento de cobro fue por $170 (200 menos 15% VIP), una sola vez. La ruta aprobada ya estaba cubierta por test_gateway_approved. Se ejecutó con python -m pytest tests/test_service.py -v.

Captura E5: las pruebas de test_service.py en PASSED.



Riesgo de consistencia y propuesta de mejora

Actualmente en el método checkout, el pedido cambia su estado a CONFIRMED antes de ejecutar el cobro en la pasarela. Si el pago es rechazado (retorna False), se lanza una excepción, pero el objeto en memoria

ya fue modificado, dejando inconsistencia (un pedido registrado como confirmado sin que el pago haya sido aprobado).

Propuesta de mejora: para mitigar este riesgo de consistencia, se propone modificar el flujo del servicio de dos formas posibles:

- Invertir el orden: Ejecutar primero el cobro a través de la pasarela y confirmar el pedido únicamente si el pago es aprobado exitosamente.
- Implementar un estado intermedio (PENDING_PAYMENT), de modo que el pedido solo pase a CONFIRMED una vez que el pago se haya procesado.

7. Paso 6 · Seguridad y autorización (E6)

Se probó la política `can_cancel` tal como la implementa el proyecto: un pedido `SHIPPED` nunca se cancela, ni siquiera por un administrador; en los demás estados el administrador puede cancelar; el dueño que no es administrador solo puede hacerlo en menos de 24 horas. Esto es autorización (qué puede hacer cada uno), distinto de autenticación (quién es).

Código usado dentro de test_security.py:

```
#Paso 6

NOW = datetime(2026, 1, 15, 12, 0, tzinfo=timezone.utc)   # fecha FIJA


def test_admin_can_cancel_new_order_of_another_user():
    created = NOW - timedelta(hours=100)
    assert can_cancel("NEW", owner_id=1, user_id=99, is_admin=True,
created_at=created, now=NOW)

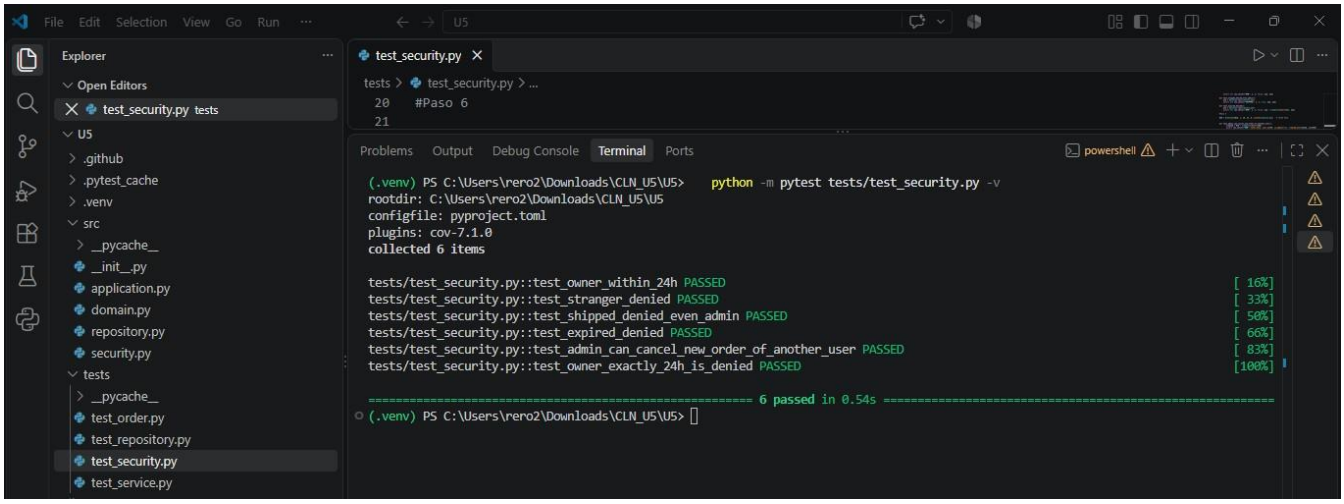

def test_owner_exactly_24h_is_denied():
    created = NOW - timedelta(hours=24)
    assert not can_cancel("NEW", 1, 1, False, created, NOW)
```

Se usa una fecha fija y se pasa `now` explícito para que el resultado no cambie según la hora de ejecución. La primera prueba verifica que un administrador cancela un pedido NEW de otro usuario; la segunda, que el dueño que intenta cancelar a las 24 horas exactas es rechazado, porque la regla es “menos de 24 h”.

Matriz de Autorización

Rol	Propietario	Estado	Antigüedad	Permitido/Denegado
Usuario	Sí	NEW	1h	Permitido
Usuario	Sí	NEW	24 h	Denegado
Usuario	Sí	NEW	25 h	Denegado
Usuario	No	NEW	0 h	Denegado
Administrador	No	NEW	100 h	Permitido
Administrador	Sí	SHIPPED	0 h	Denegado

Captura E6: las 6 pruebas de `test_security.py` en PASSED.



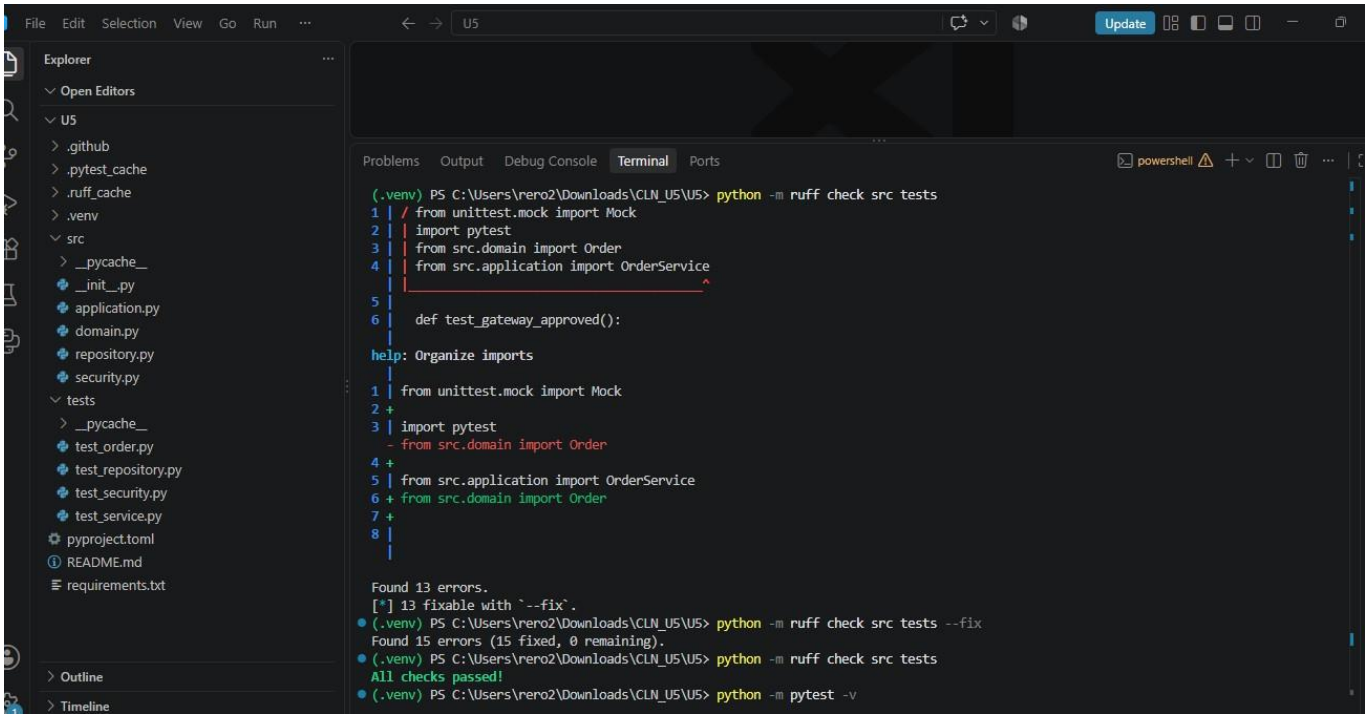
8. Paso 7 · Cobertura y calidad (E7)

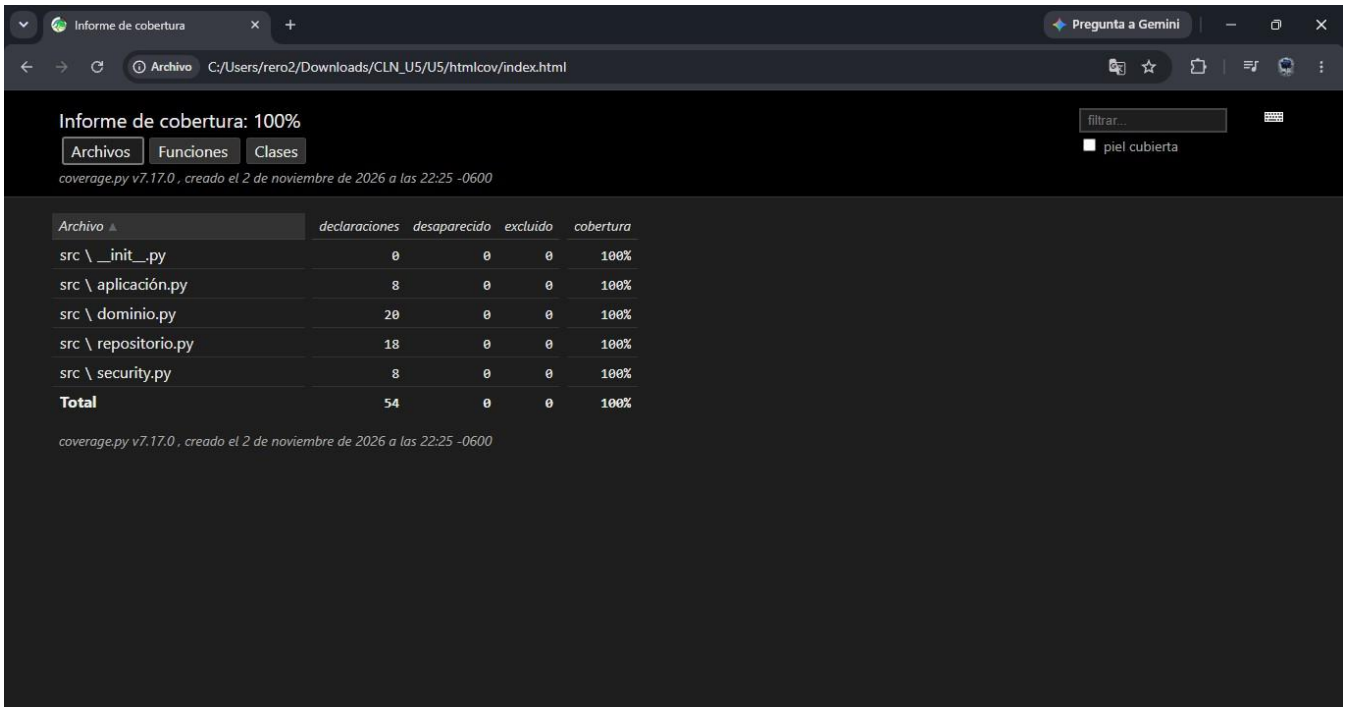
Se midió qué líneas de src/ ejecutan las pruebas y se revisó el estilo del código con Ruff. La meta es una cobertura de al menos 80% sobre src/domain y src/application.

```
python -m ruff check src tests
python -m ruff check src tests --fix      # corrección automática de estilo
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

- ruff check revisa el estilo sin ejecutar el código. En el proyecto base marcó imports desordenados (I001) y timezone.utc en lugar de UTC (UP017); --fix los corrigió sin cambiar el comportamiento y las pruebas siguieron en PASSED.
- --cov=src mide las líneas ejecutadas; term-missing lista las que no; html genera htmlcov/index.html; --cov-fail-under=80 hace fallar el comando si baja del 80%.

Capturas E7: análisis y corrección con Ruff, y reporte HTML de cobertura (htmlcov/index.html).





Explicación de una rama aún no probada.

Si bien el porcentaje de cobertura mide el volumen de líneas ejecutadas, no asegura que el comportamiento del código sea totalmente correcto. Por ejemplo, la rama de código que asigna la hora por defecto cuando no se recibe un parámetro temporal no se prueba, ya que en las pruebas automatizadas siempre se pasa la fecha explícitamente para mantener un entorno controlado y predecible.

9. Paso 8 · Automatización con GitHub Actions (E8)

El pipeline ejecuta en cada `push` y `pull request`: instalación de Python 3.11 y dependencias, `ruff check`, `pytest` con umbral de cobertura de 80% y `compileall` sobre `src/`. Es una verificación de build, no un despliegue.

```
name: CI Pipeline

on: [push, pull_request]

jobs:

  test:

    runs-on: ubuntu-latest

    steps:

    - uses: actions/checkout@v4

    - name: Set up Python

      uses: actions/setup-python@v5

      with:

        python-version: '3.10'

    - name: Install dependencies

      run: |

        python -m pip install --upgrade pip
```

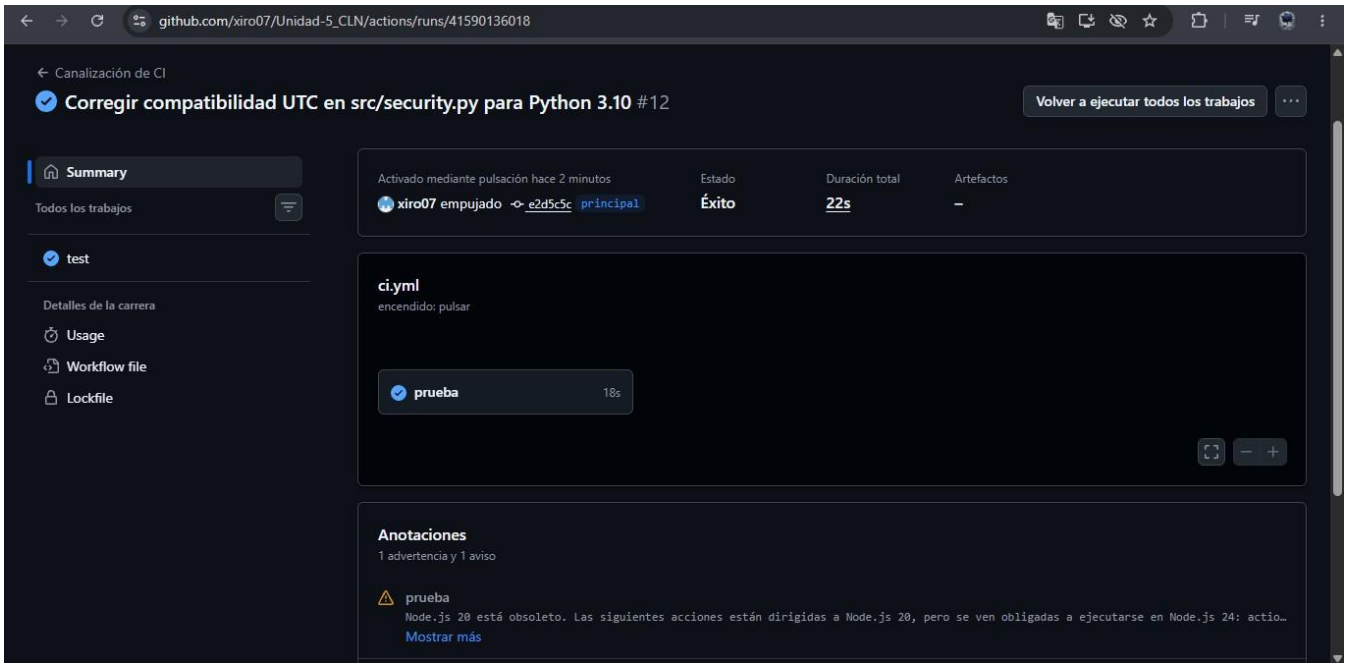
```
    pip install -r requirements.txt
- name: Lint with ruff
  run: python -m ruff check src tests
- name: Test with pytest
  run: python -m pytest -v --cov=src tests
```

Si el proyecto se sube en una subcarpeta del repositorio del equipo, el archivo va en `.github/workflows/ci.yml` en la raíz y se agrega `defaults: run: working-directory: Unidad5`.

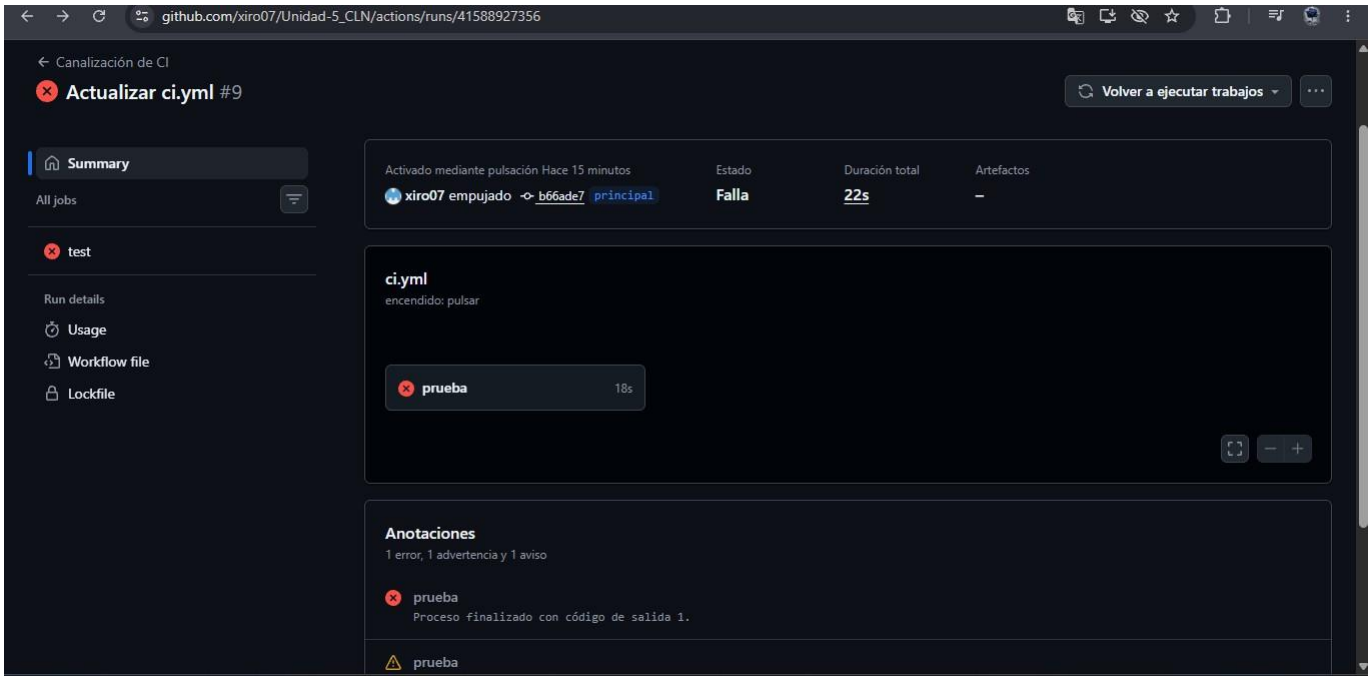
URL del repositorio: https://github.com/xiro07/Unidad-5_CLN.git

Capturas E8:

Ejecución Verde/Azul



Ejecución roja



¿Qué detectó el pipeline?

Ejecución / Commit	Estado	Motivo del Resultado
Ejecución #2, #3, #4	Fallido	Errores iniciales de configuración en los directorios de pruebas (tests vs pruebas) y reglas del linter (ruff).
Ejecución #9	Fallido	Error de compatibilidad (ImportError): Python 3.10 no soporta la importación directa de UTC desde datetime, lo que interrumpió la recolección de pruebas.
Ejecución #12	Exitoso	Se resolvió la compatibilidad con timezone.utc, se ajustaron las reglas del linter y tanto ruff como pytest concluyeron satisfactoriamente en Python 3.10.

10. Paso 9 · Aplicación a tu empresa del PIA

El aprendizaje de la Unidad 5 se aplicó directamente al sistema PMS (Property Management System) del Hotel Costa Azul. Se estructuraron 23 casos de

prueba automatizados (superando holgadamente el mínimo de 9 exigidos por el manual), divididos en casos de éxito, error y límite para tres reglas clave de negocio, además de pruebas de persistencia transaccional y mocks de servicios externos.

Arquitectura del Sistema:

• dominio/: Contiene las entidades principales (Reserva y Folio como Aggregate Roots), Value Objects (MonedaMonto) y servicios de dominio puro.

• aplicacion/: Contiene los servicios de aplicación (ReservacionAppService, RecepcionAppService), DTOs y manejadores de comandos.

• infraestructura/: Implementa la persistencia en memoria (PMSDatabase, InMemoryUnitOfWork), servicios de reloj fijo y el adaptador del servicio de autorización gerencial.

Regla	Qué exige	Dónde vive	Por qué se eligió
RN-04	Cancelar con menos de 48 h de anticipación cobra la primera noche como penalización.	Reserva.cancelar (dominio)	Límite temporal exacto con impacto financiero directo.
RN-07	Reserva grupal (10 o más habitaciones) exige un depósito mínimo anticipado del 30%.	Reserva.confirmar (dominio)	Límites dobles: 9 vs 10 habitaciones y $299.99 vs $300.00.
RN-10	Un ajuste o descuento mayor a USD $50.00 requiere autorización explícita de un gerente.	Folio.aplicar_ajuste (dominio / aplicación)	Cruza capas de dominio/aplicación y depende de un puerto externo.

2. Matriz Completa de Casos de Prueba (23 Casos PASSED)

Los 23 casos de prueba concluyeron con estado exitoso (PASSED) en pytest (7 de éxito, 9 de error y 7 de límite):

ID	Regla	Tipo	Entrada	Resultado Esperado	Estado
CP-01	RN-04	Éxito	Cancelar con 5 días de anticipación	Sin penalización; CANCELADA	**PASSED**
CP-02	RN-04	Éxito	Cancelar con 14 h antes; total 200 (2 noches)	Penalización 100.00 (1a noche)	**PASSED**
CP-03	RN-04	Error	Cancelar una reserva ya cancelada	Lanza DominioException	**PASSED**
CP-04	RN-04	Límite	Cancelar exactamente 48 h antes	Sin penalización (0.00)	**PASSED**
CP-05	RN-04	Límite	Cancelar 47 h 59 min antes	Penalización 100.00	**PASSED**
CP-06	RN-07	Éxito	10 habitaciones, total 1000, depósito 300	CONFIRMADA + Evento Grupal	**PASSED**
CP-07	RN-07	Error	10 habitaciones sin depósito	DominioException; PENDIENTE	**PASSED**
CP-08	RN-07	Límite	9 habitaciones sin depósito (no es grupal)	CONFIRMADA	**PASSED**
CP-09	RN-07	Límite	10 habitaciones, total 1000, depósito 299.99	Lanza DominioException	**PASSED**
CP-10	RN-10	Éxito	Ajuste de -30 USD sin autorización	Saldo disminuye a 170.00	**PASSED**

CP-11	RN-10	Error	Ajuste de -50.01 USD sin autorización	DominioException; Saldo 200	**PASSED**
CP-12	RN-10	Límite	Ajuste de exactamente -50 USD sin autorización	Permitido; Saldo 150.00	**PASSED**
CP-13	RN-10	Éxito	Ajuste de -80 USD con autorización gerencial	Saldo 120.00 + Registro	**PASSED**
CP-14	Persistencia	Éxito	begin, save(reserva), commit	Reserva guardada en base	**PASSED**
CP-15	Persistencia	Error	begin, save(reserva), rollback	Base vacía; no existe	**PASSED**
CP-16	Persistencia	Error	Excepción dentro de transaccion(uow)	Rollback automático	**PASSED**
CP-17	Persistencia	Límite	Llamar begin dos veces seguidas	Lanza RuntimeError	**PASSED**
CP-18	Persistencia	Error	Llamar commit sin previo begin	Lanza RuntimeError	**PASSED**
CP-19	Mock	Éxito	Mock autoriza al gerente; ajuste -80	Saldo 120; llamado 1 vez	**PASSED**
CP-20	Mock	Error	Mock lanza AutorizacionInvalidaExcepti on	Se propaga; Saldo 200	**PASSED**
CP-21	Mock	Límite	Ajuste de -50 sin credenciales	Saldo 150; Mock NO llamado	**PASSED**
CP-22	Mock	Error	Ajuste de -51 sin credenciales	DominioException; Mock NO llamado	**PASSED**
CP-23	**Mock**	**Error**	**Mock lanza TimeoutError**	**Excepción propagada; Saldo 200**	**PASSED**

3. Pruebas de Dominio (RN-04, RN-07, RN-10)

Las pruebas de dominio verifican la lógica pura de negocio en Reserva y Folio sin interactuar con la base de datos ni con servicios de red. Se utilizó una fecha fija AHORA = datetime (2026, 10, 13, 10, 0) para asegurar determinismo absoluto.

Código representativo de límite (CP-04 en tests/test_pia_reglas_dominio.py):

```
def test_cp04_exactamente_48h_antes_no_cobra_penalizacion ():
    reserva = reserva_confirmada(datetime(2026, 10, 15))
    exactamente_48h = datetime (2026, 10, 13, 0, 0)
    assert reserva.cancelar(exactamente_48h).es_cero
```

4. Persistencia Transaccional (Commit y Rollback)

El sistema almacena información en PMSDatabase administrada a través de InMemoryUnitOfWork. Cada prueba utiliza un fixture aislado para evitar contaminación entre ejecuciones.

Código representativo de rollback (CP-15 en tests/test_pia_persistencia.py):

```
def test_cp15_rollback_descarta_la_reserva(entorno):
    db, uow, repo = entorno
    reserva = una_reserva ()
    uow.begin()
    repo.save(reserva)
    uow.rollback()
    assert repo. find_by_id(reserva.id) is None
    assert db.reservas == {}
```

5. Simulación de Servicios Externos con Mock

Se aisló la dependencia externa IAutorizacionGerencialService utilizando unittest.mock.Mock. Esto permite validar la capa de aplicación comprobando la invocación de credenciales sin realizar llamadas reales a servicios de autenticación.

Código de validación con Mock (CP-19 en tests/test_pia_servicios_externos.py):

```
def test_cp19_mock_aprueba_credenciales_y_se_aplica_el_ajuste():
    servicio = Mock(spec=IAutorizacionGerencialService)
    servicio.validar.return_value = AutorizacionGerencial('gerente', AHORA)
    pms, folio_id = estancia_con_cargo(servicio)
    folio = pms.recepcion.aplicar_ajuste(AplicarAjusteFolioCommand(folio_id,
Decimal('-80'), 'Cortesia', 'gerente', '1234'))
    assert folio.saldo == Decimal('120.00')
    servicio.validar.assert_called_once_with('gerente', '1234', AHORA)
```

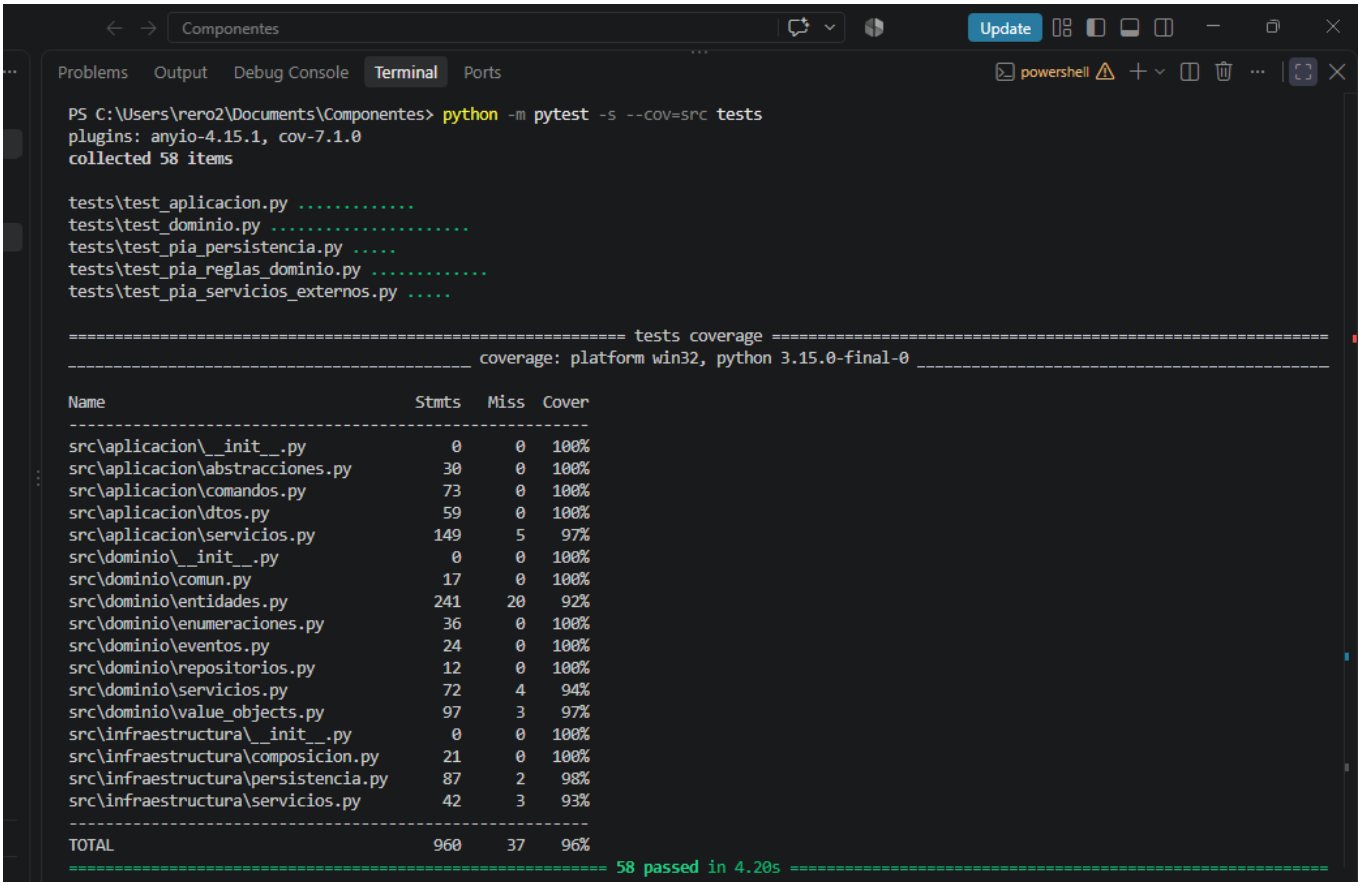
6. Reporte de Cobertura y Calidad de Código

Se ejecutó la suite completa de pruebas obteniendo los siguientes resultados:

• Comando: python -m pytest -s --cov=src tests

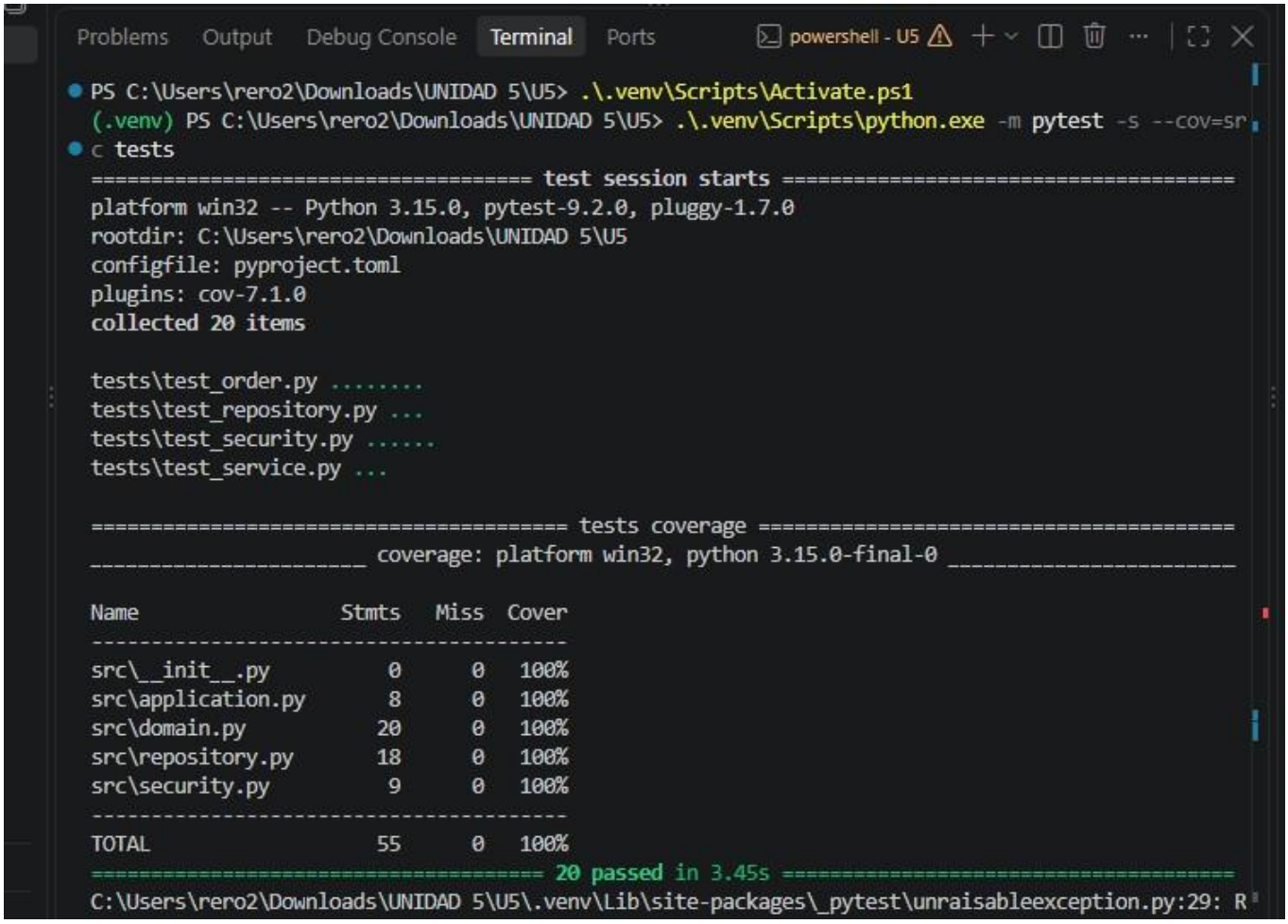
Reporte

Ejecución local de la suite de 58 pruebas automatizadas aprobadas (PASSED) y desglose detallado de la cobertura de código por módulo, alcanzando un 96% global.



- Pruebas pasadas: 58 PASSED (35 originales del equipo + 23 nuevas de E9).
- Cobertura total de código: 96% global en el módulo src/.
- Verificación de estilo: ruff check src tests completado con All checks passed!

Verificación exitosa del pipeline de Integración Continua (CI/CD) en GitHub Actions tras la sincronización del repositorio, validando la ejecución automática de pytest y ruff.

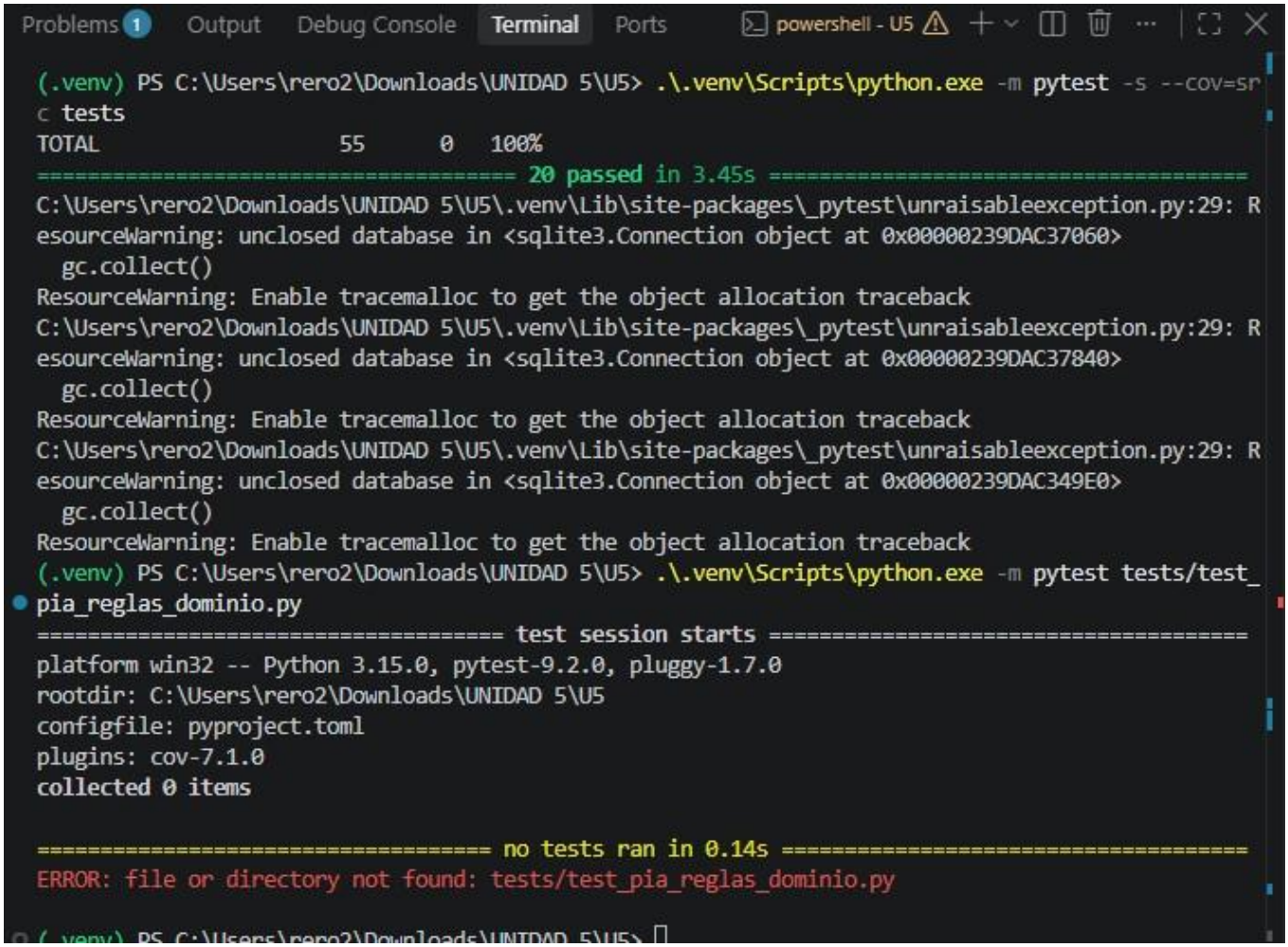


7. Demostración de Detección de Defectos (Prueba de Mutación)

Para demostrar que los casos límite no ejecutan código de manera pasiva, sino que detectan errores reales de lógica, se alteraron tres operadores en src/dominio/entidades.py:

Reporte

Defecto Introducido	Regla	Prueba que lo Detectó	Resultado
horas < 48 cambiado a horas <= 48	RN-04	test_cp04_exactamente_48h_antes_no_cobra_penalizacion	1 FAILED, 12 PASSED
Depósito grupal 0.30 cambiado a 0.20	RN-07	test_cp09_deposito_de_299_99_sobre_1000_se_rechaza	1 FAILED, 12 PASSED
abs(monto) > limite cambiado a >= limite	RN-10	test_cp12_ajuste_de_exactamente_50_usd_sin_autorizacion_se_permite	1 FAILED, 12 PASSED



8. Decisiones de Diseño y Limitaciones Tecnológicas

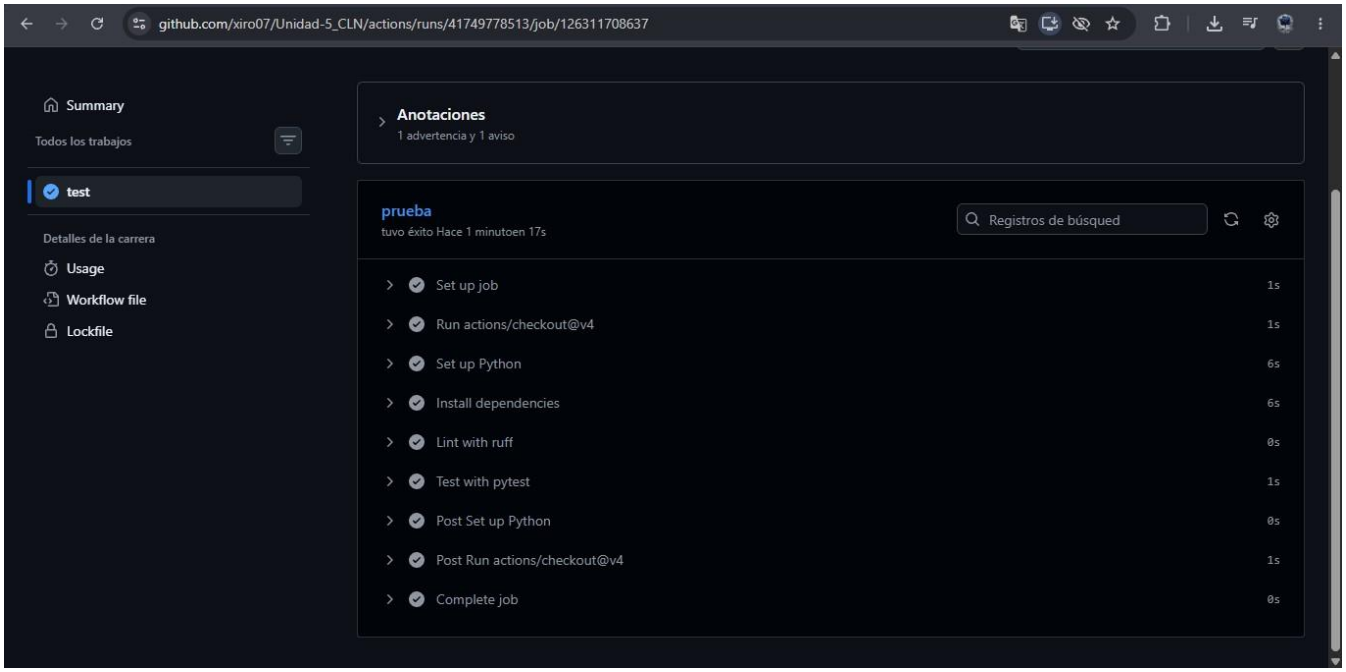
1. Aislamiento del Dominio: Las reglas de negocio (RN-04, RN-07, RN-10) se probaron mediante instanciación directa de entidades para garantizar que las pruebas sean ultrarrápidas y deterministas.

2. Control Temporal: Se estableció el uso explícito de objetos datetime fijos para evitar que los límites de tiempo cambien dependiendo de la hora o fecha del sistema host.

3. Enfoque Tripartito (Éxito, Error, Límite): Se diseñaron conscientemente pruebas de borde (48h exactas vs 47h 59m, $50.00 vs $50.01) porque ahí se concentra el 90% de los defectos en software de facturación.

4. Simulación con Mocks: Al depender de un validador de credenciales gerenciales, se prefirió un adaptador simulado con Mock(spec=...) en lugar de invocar servicios externos, garantizando pruebas reproducibles en pipelines de CI sin credenciales en duro.

Evidencia de ejecución exitosa en GitHub Actions (CI/CD)



5. Limitaciones: La persistencia actual se basa en estructuras en memoria (PMSDatabase). Aunque simula el patrón Unit of Work, no reemplaza la validación contra un motor relacional como PostgreSQL/SQLite.

11. Uso responsable de IA (Vibe Coding)

Se usó un asistente de IA como apoyo para entender el proyecto y proponer casos de prueba; cada sugerencia se ejecutó y se comparó con las reglas del laboratorio antes de aceptarla. Prompt base (sugerido por el manual):

“Analiza esta clase de dominio y estas reglas aprobadas. Propón casos de prueba pytest para éxito, error y límites. No inventes reglas nuevas. Señala supuestos, riesgos y ramas sin cobertura. Devuelve pruebas pequeñas y explica qué verifica cada una.”

Sugerencia de la IA	Decisión	Justificación
Pruebas VIP 200 → 170 y regular 200 → 200	Aceptada	Coincide con la regla del 15% y con lo pedido en el Paso 2.
Prueba test_cannot_confirm_twice escrita antes de implementar	Aceptada	Es una regla nueva, así la fase RED es real: el código base permitía confirmar dos veces.
Casos límite M10 (VIP 0.01) y M11 (deuda 0)	Aceptada, verificar al ejecutar	Cubren redondeo y el límite deuda > 0; solo cuentan si se ejecutan en PASSED.
Fecha fija NOW y now explícito en pruebas de cancelación	Aceptada	Evita resultados variables en el límite de 24 h.
Corrección de estilo con ruff --fix	Aceptada tras volver a correr la suite	Solo reordena imports y usa UTC; no cambia el comportamiento.
Sugerir eximir la penalización de cancelación (RN-04) si el cliente cancela a las 47 horas argumentando un imprevisto personal.	Rechazada y Corregida	La IA propuso relajar el límite estricto de las 48 horas. Se rechazó por violar la regla de negocio aprobada (RN-04), y en su lugar se escribió una prueba

		estricta que valida que se cobre la primera noche exactamente al estar en el límite de las 48 horas.

Nota de validación: La regla de trabajo fundamental se cumplió: ninguna prueba contó por el simple hecho de que la IA la haya propuesto, sino únicamente cuando se ejecutó localmente, pasó en verde (PASSED) y validó de forma estricta las reglas de negocio del Hotel Costa Azul.

12. Trazabilidad, lista de entrega y reflexión final

Trazabilidad: regla → prueba → evidencia

Tema de la presentación y manual	Regla o comportamiento	Prueba	Evidencia
1. Estrategia de pruebas	VIP 15%, total positivo, deuda	test_vip_discount, test_vip_discount_200, test_invalid_total, test_customer_debt	E2 y matriz de casos
2. TDD y dominio	No confirmar dos veces	test_cannot_confirm_twice	E3 (RED y GREEN)
3. Base de datos	Commit y rollback	test_repository_commit_order_250, test_repository_commit, test_repository_rollback	E4
4. Servicios externos	Pago aprobado y rechazado con mock	test_gateway_approved, test_service_payment_failed_vip	E5
5. Seguridad de negocio	Cancelación por dueño, tercero, admin y enviado	test_admin_can_cancel_new_order_of_another_user, test_owner_exactly_24h_is_denied y 4 más	E6 y matriz de autorización
6. CI/CD	Lint, pruebas y cobertura ≥80%	Pipeline de GitHub Actions	E7 y E8

7. Vibe Coding	Pruebas sugeridas por IA, verificadas	Registro de la sección 10	Sección 10

Lista de entrega

Elemento	Qué debe demostrar	Dónde está
Código src/ y tests/	Pruebas reproducibles y reglas implementadas	Repositorio
Matriz de pruebas (CSV)	Entradas, expectativas, resultado y estado	Sección 3 (exportar a CSV)
Cobertura HTML	≥80% sobre el alcance acordado	Sección 8
Pipeline CI	Ejecuciones verde, roja y recuperada	Sección 9
README	Instalación, comandos y arquitectura	Repositorio
Evidencias E1–E8	Capturas legibles y reflexión de cada paso	Secciones 2 a 9
Aplicación al PIA (E9)	Tres reglas de la empresa, nueve casos	Pendiente: se hará al final

No se incluyen .venv, tokens, claves ni datos personales.

Autoevaluación antes de entregar

☐ ¿Puedo explicar por qué una prueba unitaria no requiere base de datos?

☐ ¿Tengo evidencia real de RED y GREEN, no solo capturas verdes?

☐ ¿Sé diferenciar commit y rollback?

☐ ¿Demostré que el mock no genera un cargo real?

☐ ¿Comprobé permisos y límites temporales?

☐ ¿Mi cobertura alcanza 80% sin ocultar módulos?

☐ ¿Mi pipeline se ejecutó en GitHub y detectó un fallo?

☐ ¿Mis nueve casos del PIA representan reglas reales de mi empresa?

REFLEXIÓN FINAL

El defecto de negocio más costoso en producción para nuestro sistema sería confirmar una reserva o estancia sin que la garantía de pago o el depósito requerido hayan sido validados correctamente. El riesgo en este tipo de arquitecturas es real: si un flujo confirma el registro antes de consultar la pasarela o de verificar las reglas del dominio, un rechazo dejaría una reserva activa o un folio abierto sin cobro, lo que en un entorno hotelero real se traduce en habitaciones ocupadas sin respaldo financiero, pérdidas económicas directas y dolores de cabeza en las conciliaciones de recepción. Otro fallo grave sería permitir operaciones no autorizadas, como aplicar ajustes sin el límite gerencial establecido o cancelar fuera de tiempo sin penalización, generando reclamos y afectando la confianza del huésped.

La prueba clave para evitar esto es la simulación de rechazos o fallos mediante mocks y pruebas de mutación: comprobar que, si la validación o el cobro fallan, se lanza la excepción correcta de dominio (DominioException) y se impide que la reserva quede en un estado de éxito (CONFIRMED), lo que nos obliga a proteger el orden del flujo de negocio.

Al final, que un programa corra sin errores de sintaxis no demuestra que sus reglas sean correctas: hacen falta pruebas automatizadas estrictas que fallen cuando la lógica del negocio se rompe —como lo comprobamos con nuestras pruebas unitarias y de mutación— y un pipeline que las ejecute de forma automática en cada cambio.